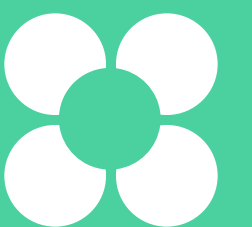


# Вёрстка форм

Владимир Языков  
Frontend-разработчик



# Владимир Языков

Frontend-разработчик

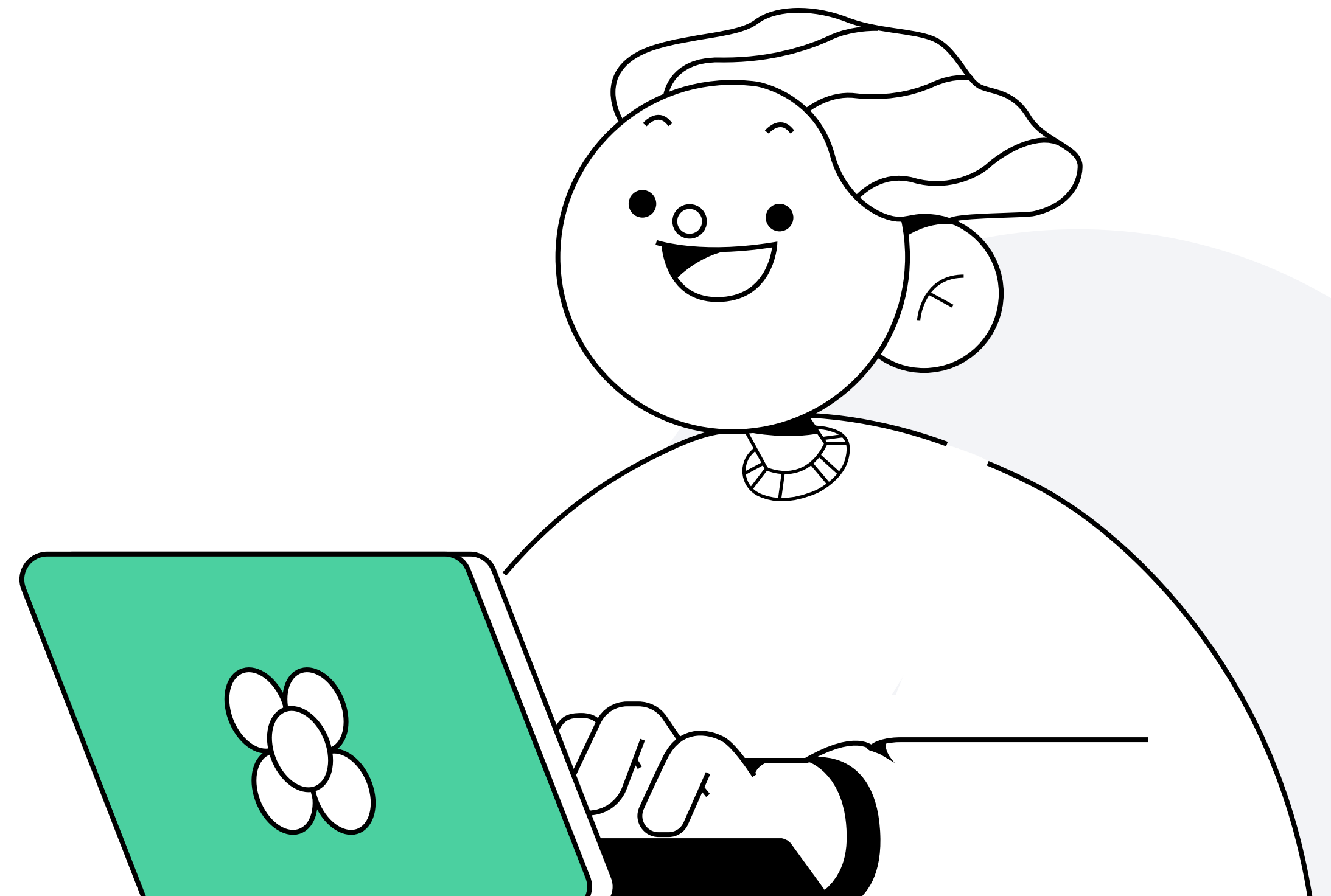


# Вёрстка форм. Создание полей для ввода

Владимир Языков  
Frontend-разработчик

# Цели занятия

- 1 Научиться создавать формы
- 2 Научиться выводить текст с примером вводимых данных в форме



# Задача

Сверстаем форму отправки сообщения

Введите ваше имя

Например, Иван

Введите ваш email

Например, ivan@gmail.com

Введите ваше сообщение

Здравствуйте...

Отправить

# Элемент input

Элемент input позволяет создавать различные элементы формы. Для этого существует атрибут type

```
<input type="text">  
<input type="password">
```

- type="text" — элемент input становится полем для ввода текста
- type="password" — элемент input становится текстовым полем ввода для пароля

# Создание разметки

```
<div class="form-row">
  <span class="form-hint">Введите ваше имя</span>
  <input type="text" class="field">
</div>
<div class="form-row">
  <span class="form-hint">Введите ваш email</span>
  <input type="text" class="field">
</div>
```

Введите ваше имя

Введите ваш email

# Стилизация подсказки

```
.form-hint {  
  display: block;  
  font-size: 13px;  
  font-weight: 700;  
  margin-bottom: 12px;  
}
```

**Введите ваше имя**

**Введите ваш email**

# Стилизация полей

```
.field {  
  box-sizing: border-box;  
  width: 640px;  
  border: 1px solid #222222;  
  padding: 15px 20px;  
}
```

Введите ваше имя

Введите ваш email

# Атрибут placeholder

Предназначен для вывода текста внутри поля формы, чтобы помочь пользователю ввести верные данные

```
<div class="form-row">
  <span class="form-hint">Введите ваше имя</span>
  <input type="text" class="field" placeholder="Например, Иван">
</div>
<div class="form-row">
  <span class="form-hint">Введите ваш email</span>
  <input type="text" class="field" placeholder="Например, ivan@gmail.com">
</div>
```

Введите ваше имя

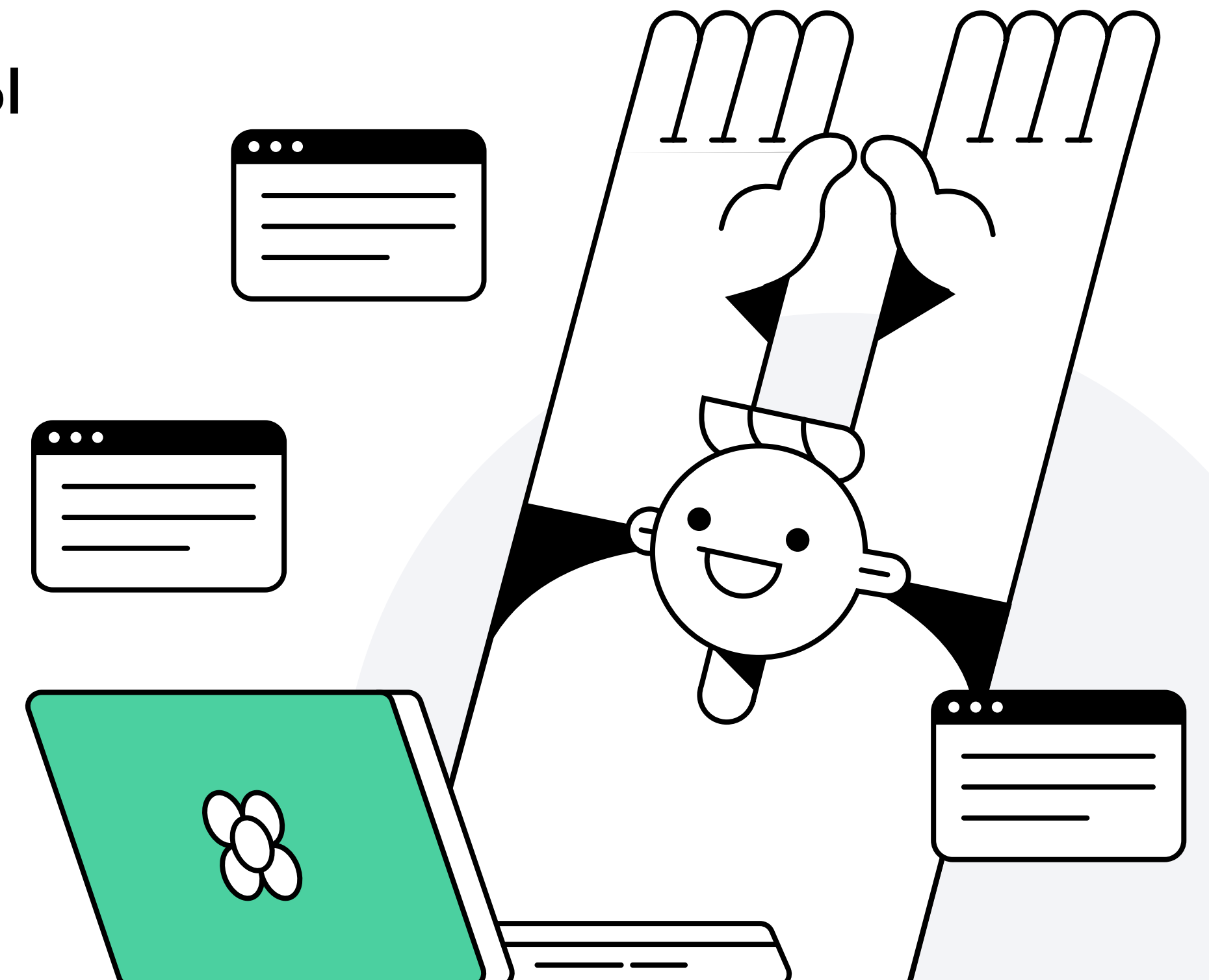
Например, Иван

Введите ваш email

Например, ivan@gmail.com

# Итоги

- 1 Элемент `input` позволяет создавать элементы формы, благодаря различным значениям атрибута `type`
- 2 Атрибут `placeholder` предназначен для вывода текста внутри поля формы

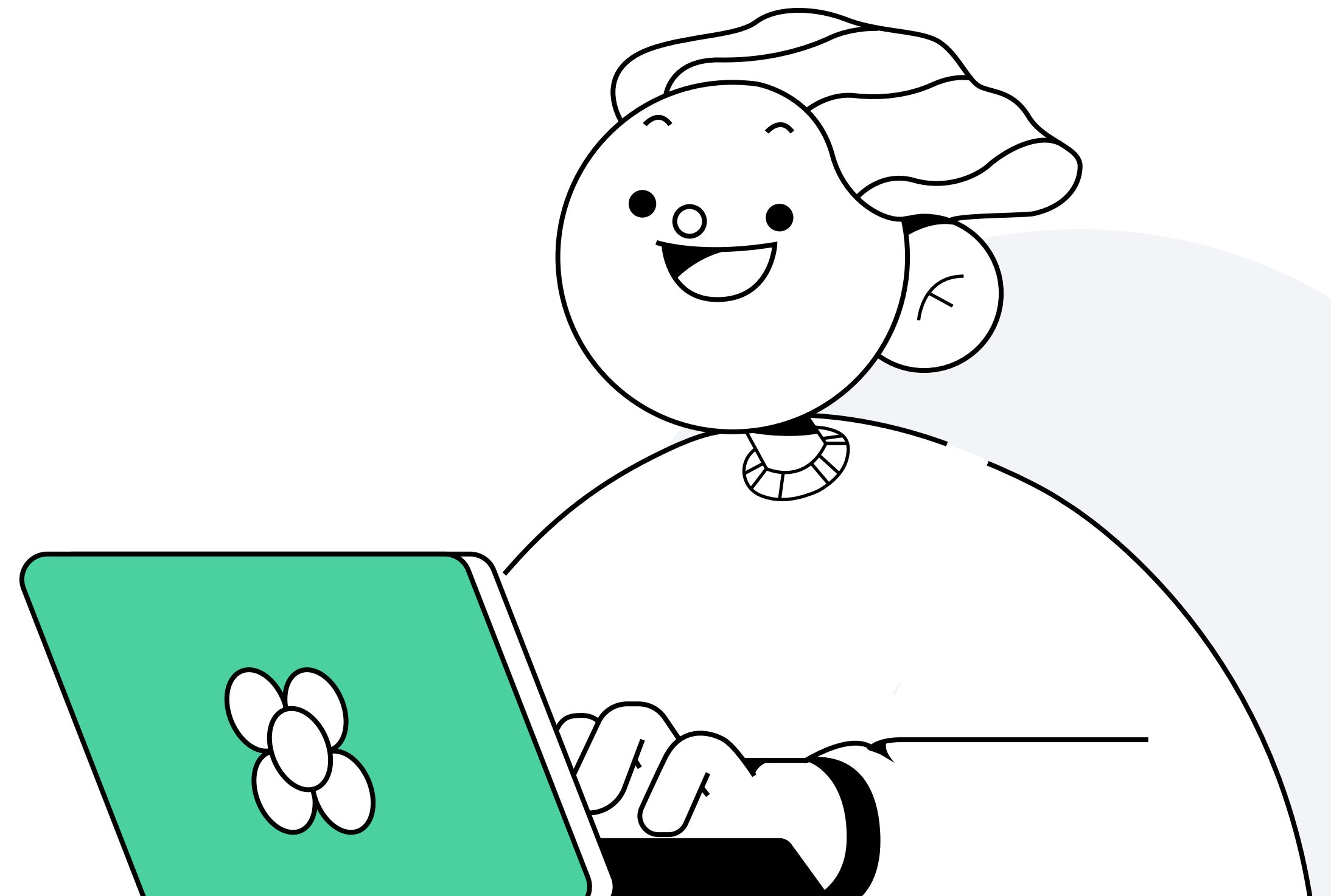


# Вёрстка форм. Размер контентной области

Владимир Языков  
Frontend-разработчик

# Цели занятия

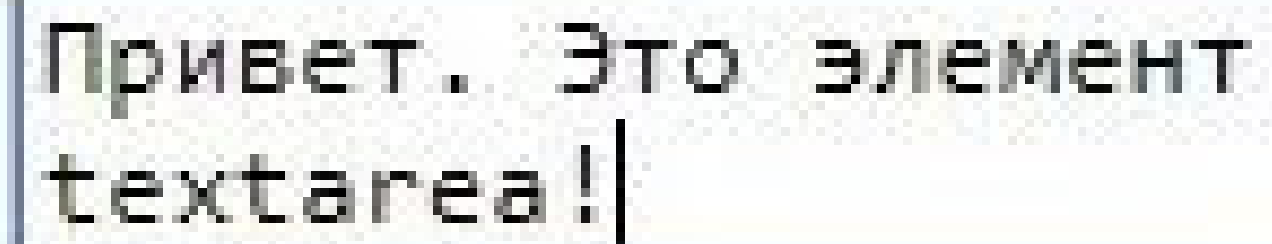
- 1 Научиться создавать поле для нескольких строк текста
- 2 Научиться регулировать размер контентной области формы



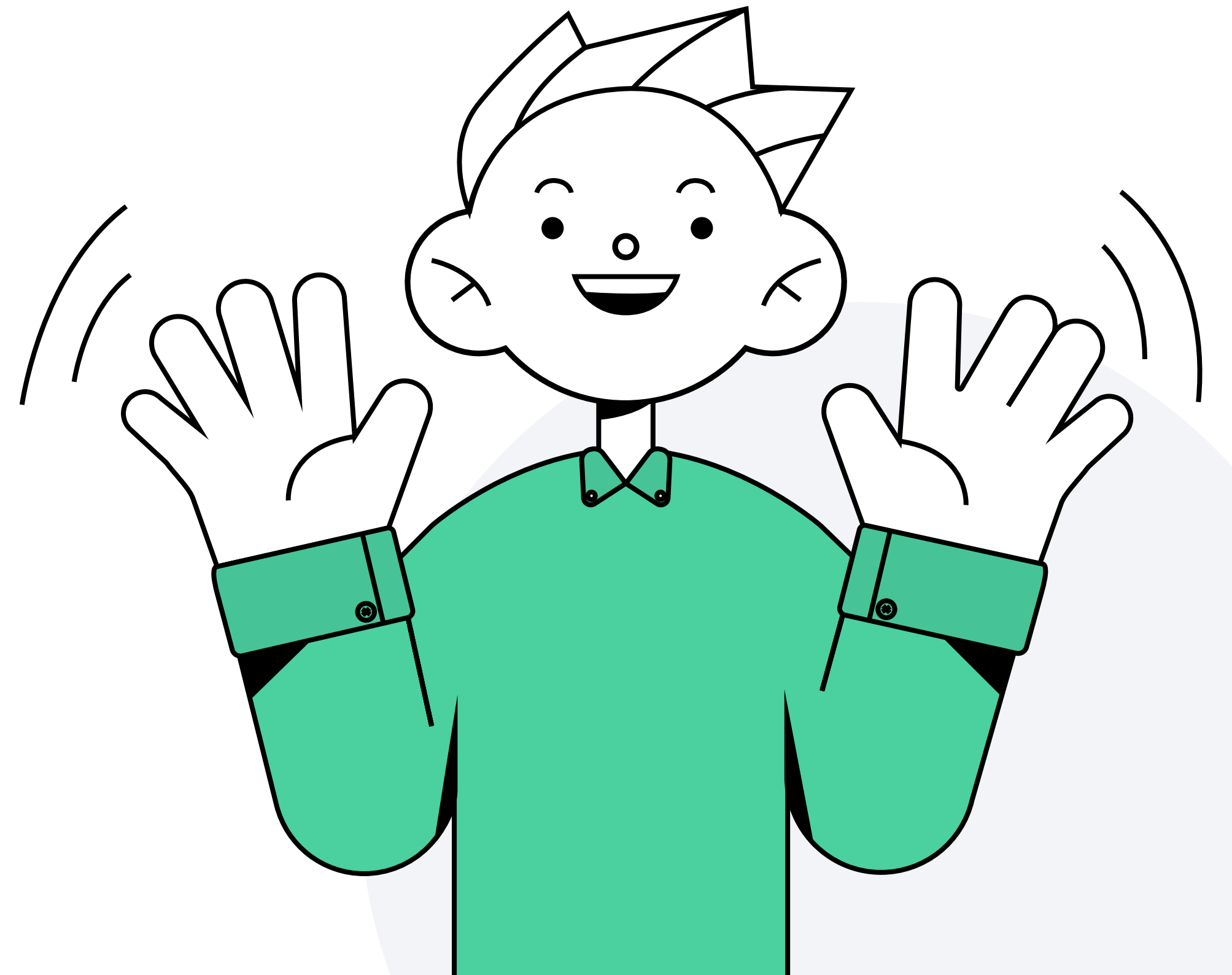
# Элемент textarea

Элемент формы для создания области, в которую  
МОЖНО ВВОДИТЬ несколько строк текста

```
<textarea></textarea>
```



Привет. Это элемент  
textarea!



# Элемент textarea

```
<div class="form-row">  
  <span class="form-hint">Введите ваше сообщение</span>  
  <textarea class="field" placeholder="Здравствуйтесь..."></textarea>  
</div>
```

Введите ваше имя

Например, Иван

Введите ваш email

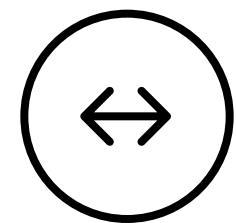
Например, ivan@gmail.com

Введите ваше сообщение

Здравствуйтесь...

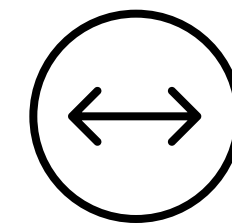
# Размер контентной области

В CSS есть свойства, которые контролируют диапазон изменения ширины и высоты контентной области, то есть свойств `width` и `height`



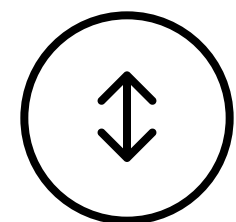
## `min-width`

Задаёт минимальное значение, которое может принимать свойство `width`



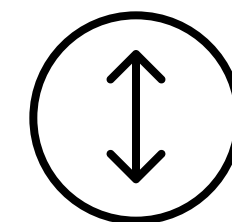
## `max-width`

Задаёт максимальное значение, которое может принимать свойство `width`



## `min-height`

Задаёт минимальное значение, которое может принимать свойство `height`



## `max-height`

Задаёт максимальное значение, которое может принимать свойство `height`

# Что больше

```
.box {  
  width: 700px;  
  min-width: 1000px;  
}
```

Ширина контентной области элемента берётся из свойства min-width (1000px)

Тот же алгоритм с высотой контентной области

```
.box {  
  height: 700px;  
  min-height: 1000px;  
}
```

!

$\text{width} / \text{height} < \text{min-width} / \text{min-height}$

# Что больше

```
.box {  
  width: 800px;  
  min-width: 800px;  
}
```

Браузер берёт значение из свойства width (800px)

Тот же алгоритм с высотой контентной области

```
.box {  
  height: 800px;  
  min-height: 800px;  
}
```

!

$\text{width} / \text{height} = \text{min-width} / \text{min-height}$

# Что больше

```
.box {  
  width: 700px;  
  min-width: 500px;  
}
```

Ширина контентной области = значению из свойства width (700px)

Тот же алгоритм с высотой контентной области

```
.box {  
  height: 700px;  
  min-height: 500px;  
}
```

!

$\text{width} / \text{height} > \text{min-width} / \text{min-height}$

# Что больше

```
.box {  
  width: 700px;  
  max-width: 1000px;  
}
```

Ширина контентной области элемента берётся из width (700px)

Тот же алгоритм с высотой контентной области

```
.box {  
  height: 700px;  
  max-height: 1000px;  
}
```

!

$\text{width} / \text{height} < \text{max-width} / \text{max-height}$

# Что больше

```
.box {  
  width: 800px;  
  max-width: 800px;  
}
```

Браузер возьмёт значение из свойства width, то есть 800px

Тот же самый алгоритм сработает в случае высоты контентной области

```
.box {  
  height: 800px;  
  max-height: 800px;  
}
```

!

$\text{width} / \text{height} = \text{max-width} / \text{max-height}$

# Что больше

```
.box {  
  width: 700px;  
  max-width: 500px;  
}
```

Ширина контентной области = значение из свойства max-width (500px)

То же с высотой контентной области

```
.box {  
  height: 700px;  
  max-height: 500px;  
}
```

!

$\text{width} / \text{height} > \text{max-width} / \text{max-height}$

# Размеры textarea

Добавляем отдельный класс textarea для элемента

```
<textarea class="field textarea" placeholder="Здравствуйте..."></textarea>
```

```
.textarea {  
  min-width: 640px;  
  max-width: 640px;  
  min-height: 150px;  
  max-height: 200px;  
}
```

# Результат

Введите ваше имя

Например, Иван

Введите ваш email

Например, ivan@gmail.com

Введите ваше сообщение

Здравствуйте...

# Итоги

- 1 Элемент `textarea` предназначен для создания области, в которую можно вводить несколько строк текста
- 2 Свойства `min-width`, `max-width`, `min-height` и `max-height` контролируют диапазон изменения ширины и высоты контентной области

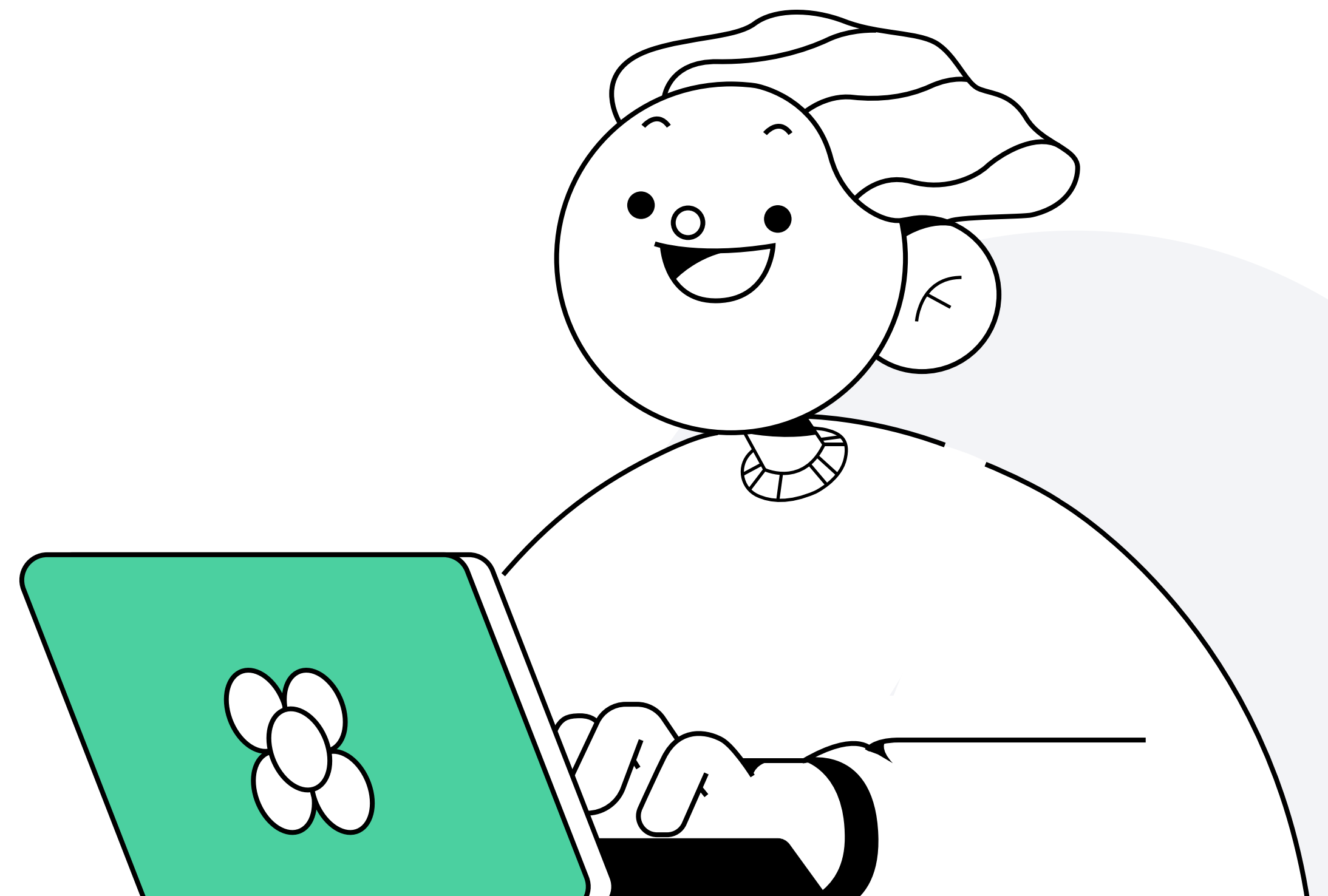


# Вёрстка форм. Наследование

Владимир Языков  
Frontend-разработчик

# Цель занятия

- 1 Узнать, как оптимизировать работу, избежав копирования схожих элементов

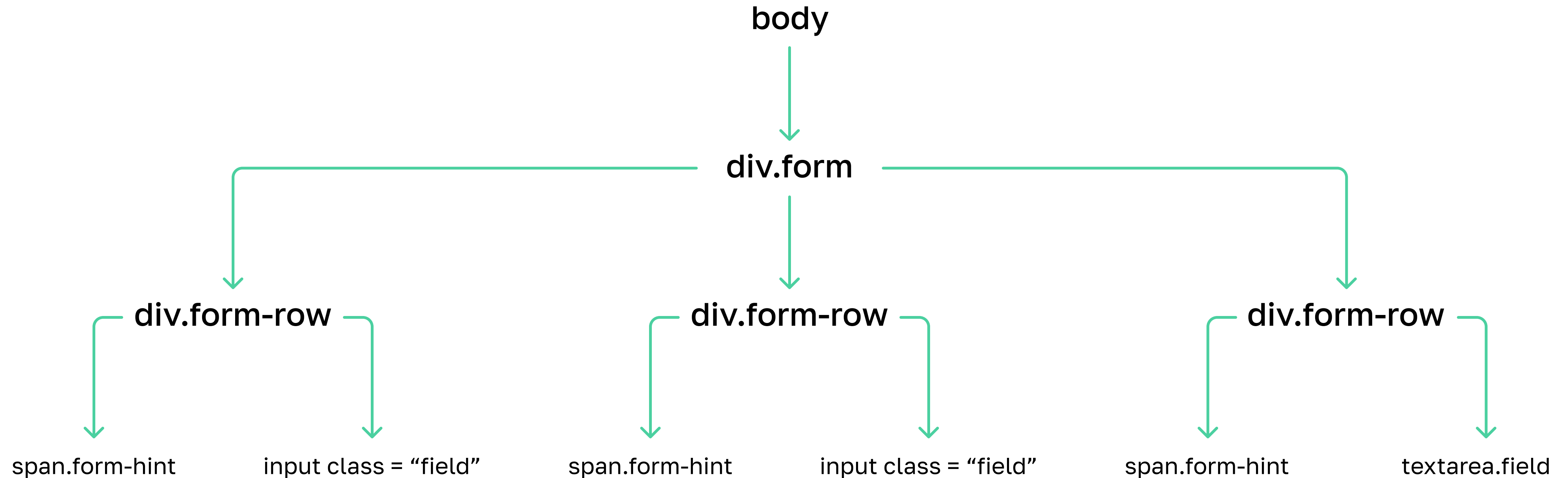


# Значение inherit

У нас есть HTML

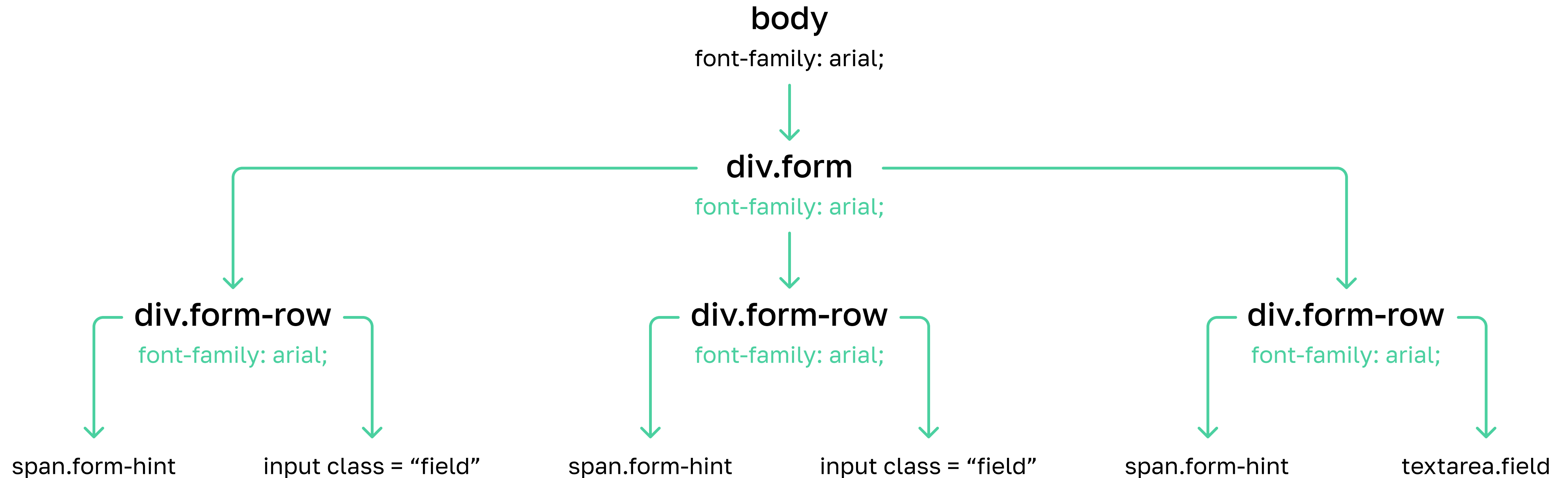
```
<body>
  <div class="form">
    <div class="form-row">
      <!-- поле для ввода имени -->
    </div>
    <div class="form-row">
      <!-- поле для ввода email -->
    </div>
    <div class="form-row">
      <!-- поле для ввода сообщения -->
    </div>
  </div>
</body>
```

# Дерево элементов



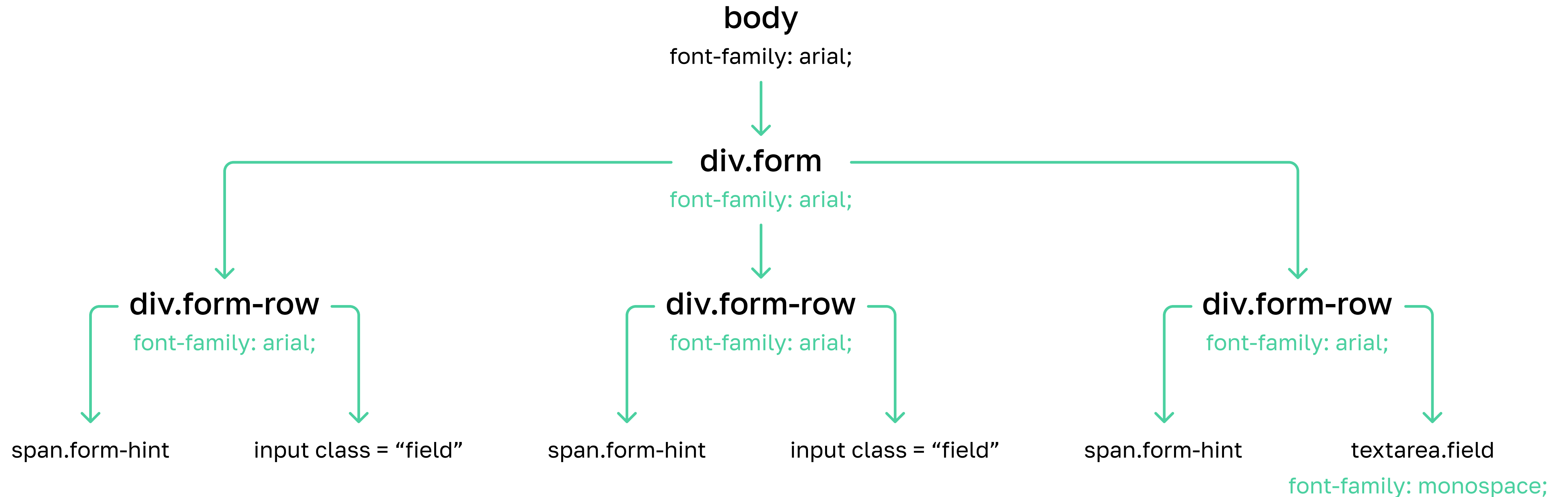
У элемента `span` с классом `form-hint` есть один родитель — элемент `div` с классом `form-row`

# Дерево элементов



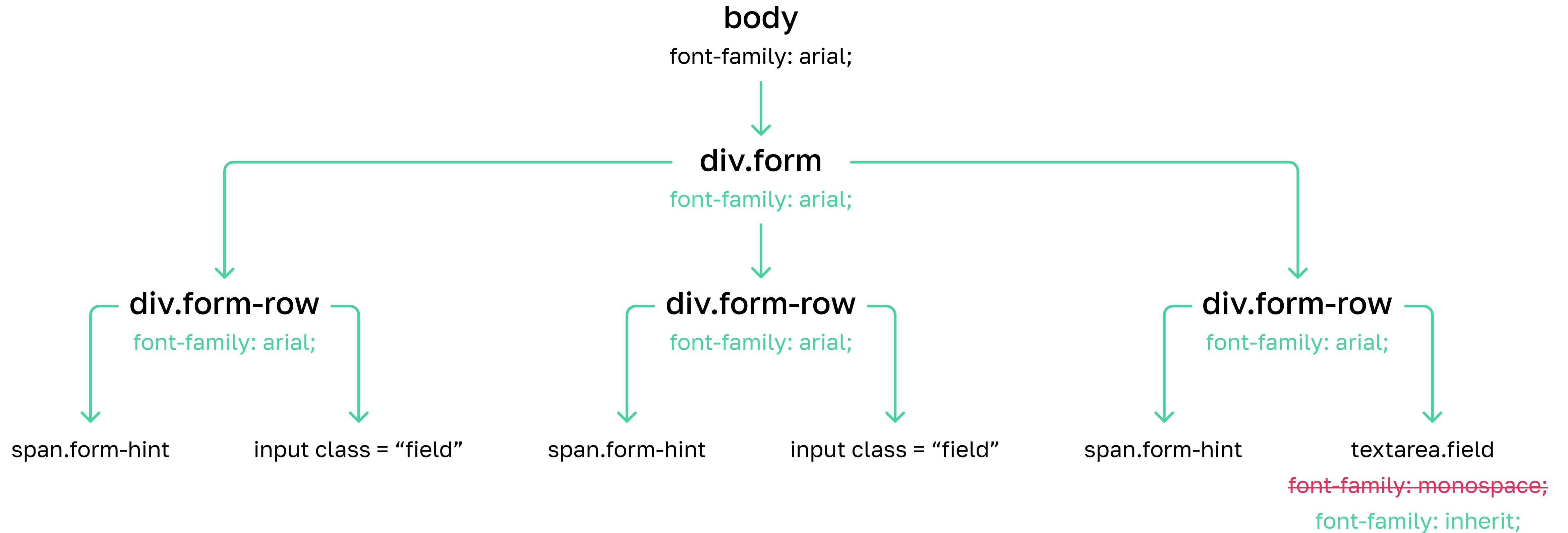
Это значение применяется не только для самого элемента, но и для его детей и потомков

# Наследование стиля



У браузера есть собственные стили,  
и они также вносят коррективы

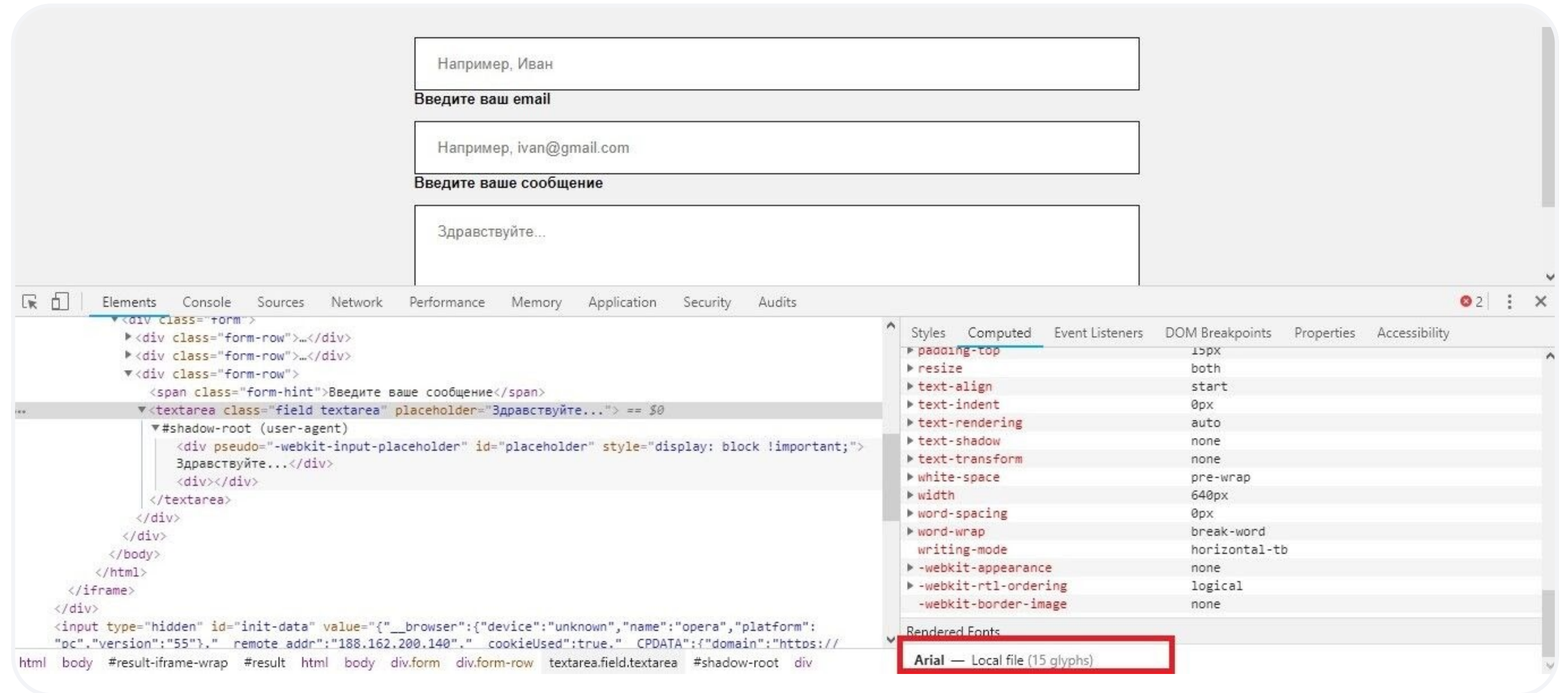
# Наследование стиля



В CSS есть возможность сказать браузеру, что ему нужно наследовать значение свойства от родителей

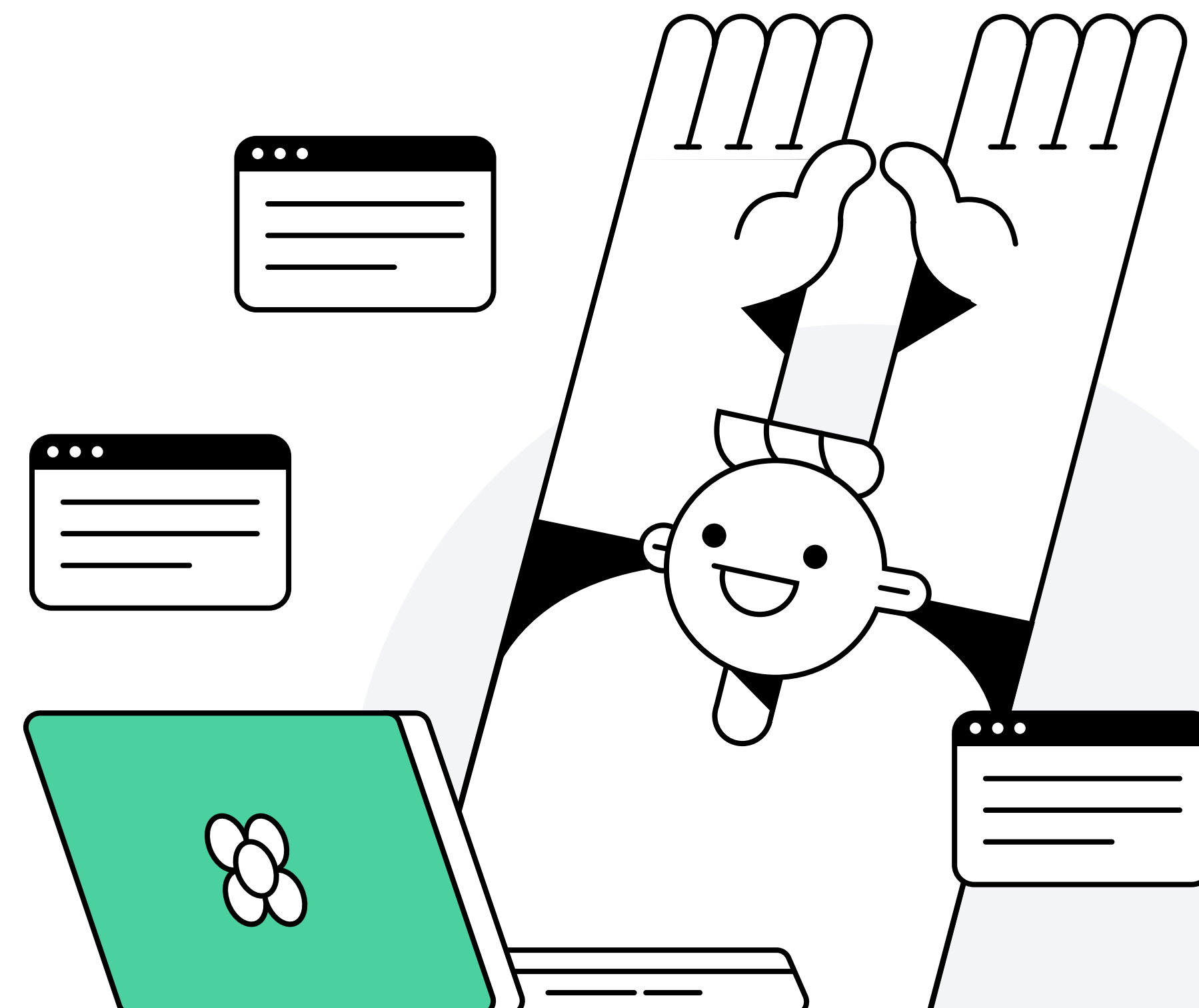
# Добавление inherit для font-family

```
.field {  
  box-sizing: border-box;  
  width: 640px;  
  border: 1px solid #222222;  
  padding: 15px 20px;  
  font-family: inherit;  
}
```



# Итоги

- 1 Свойство `inherit` позволяет наследовать стили элементов

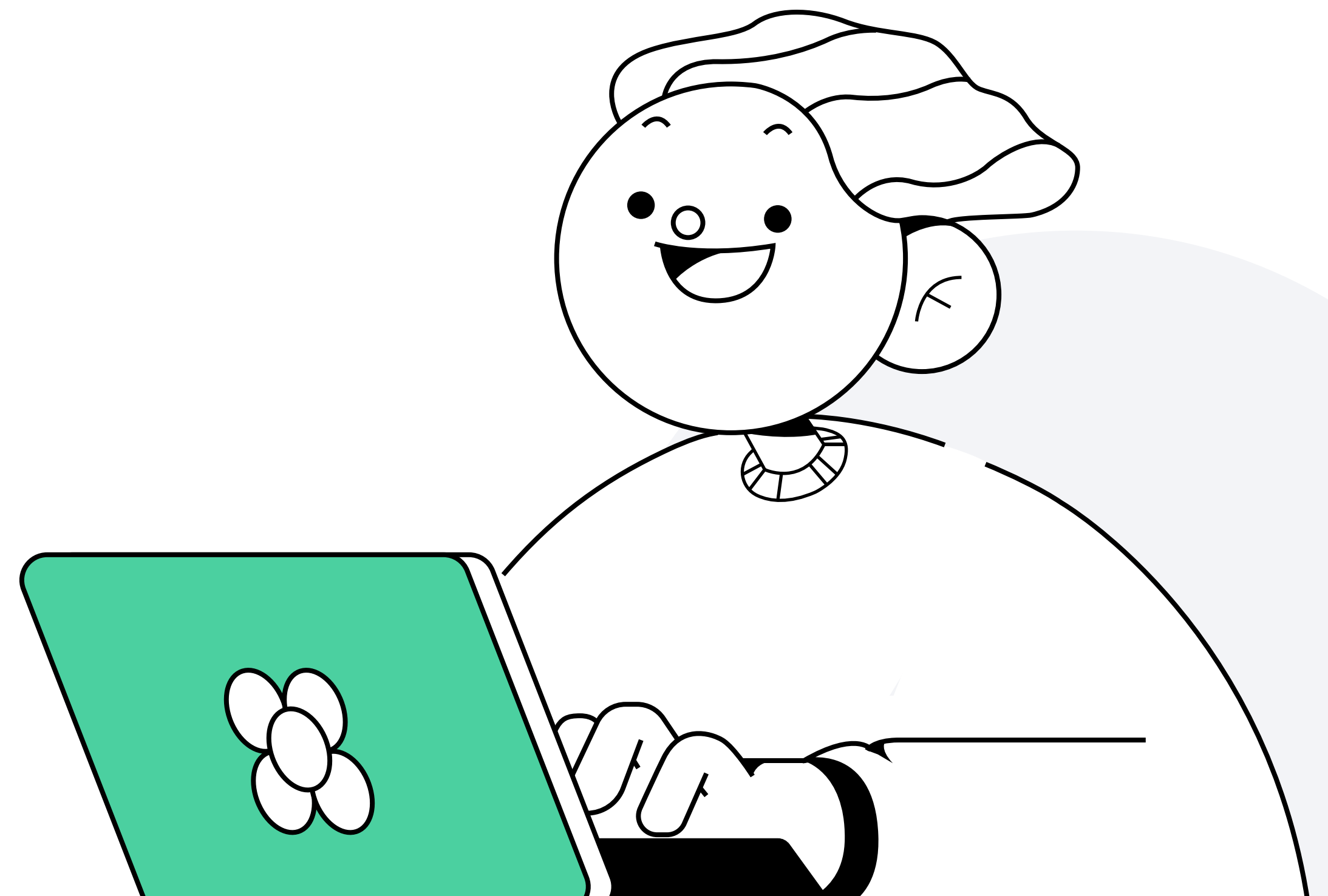


# Вёрстка форм. Кнопки

Владимир Языков  
Frontend-разработчик

# Цели занятия

- 1 Узнать, как создавать элемент «кнопка»
- 2 Узнать, как менять стиль курсора



# Элемент textarea

```
<button type="submit">Отправить</button>
```

Элемент управления, позволяющий пользователю запускать определённые действия

# Элемент button

```
<button type="submit">Отправить</button>
```

Элемент управления, позволяющий пользователю запускать определённые действия

## Типы кнопок

submit

Кнопка отправки формы.  
Значение по умолчанию

reset

Кнопка, очищающая  
значение в форме

button

Кнопка, не запускающая  
никакого действия

# Добавление кнопки

```
<div class="form-row">  
  <button class="button">Отправить</button>  
</div>
```

## Стилизуем её

```
.button {  
  border: none;  
  padding: 15px 55px;  
  color: #ffffff;  
  background-color: #e74c3c;  
  font-family: inherit;  
  font-size: 16px;  
}
```



# Результат

Введите ваше имя

Например, Иван

Введите ваш email

Например, ivan@gmail.com

Введите ваше сообщение

Здравствуйте...

Отправить

# СВОЙСТВО cursor

Существующие значения:

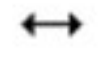
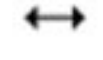
→ default

→ text

→ none

→ pointer

## CSS Custom Cursors

|                                                                                                |                                                                                                    |                                                                                                 |                                                                                                 |
|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
|  auto       |  move           |  no-drop     |  col-resize  |
|  all-scroll |  pointer        |  not-allowed |  row-resize  |
|  crosshair  |  progress       |  e-resize    |  ne-resize   |
|  default    |  text           |  n-resize    |  nw-resize   |
|  help      |  vertical-text |  s-resize   |  se-resize  |
|  inherit  |  wait         |  w-resize  |  sw-resize |

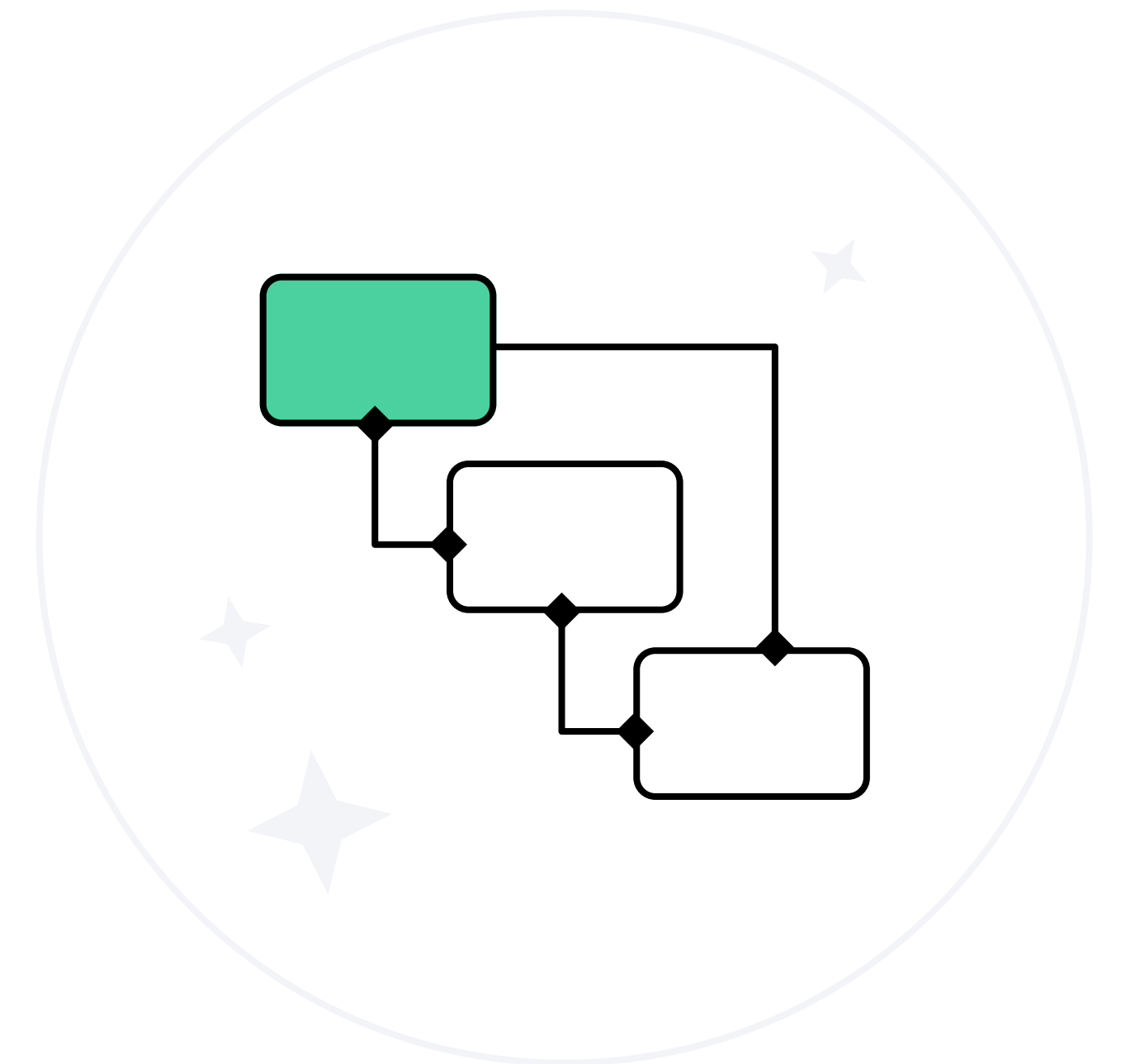
# Добавление cursor со значением pointer

```
.button {  
  border: none;  
  padding: 15px 55px;  
  color: #ffffff;  
  background-color: #e74c3c;  
  font-family: inherit;  
  font-size: 16px;  
  cursor: pointer;  
}
```



# Структура

```
<div class="form">  
  <div class="form-row">  
    <!-- поле для ввода имени -->  
  </div>  
  <div class="form-row">  
    <!-- поле для ввода email -->  
  </div>  
  <div class="form-row">  
    <!-- поле для ввода сообщения -->  
  </div>  
  <div class="form-row">  
    <!-- кнопка -->  
  </div>  
</div>
```



# Одинаковый отступ между элементами

The diagram illustrates a form layout with four main components, each enclosed in a red border. The components are stacked vertically, and the spacing between them is consistently 35px, as indicated by the double-headed arrows and '35px' labels on the left. The components are:

- Input 1:** A text input field with the placeholder text 'Введите ваше имя' and the example text 'Например, Иван'.
- Input 2:** A text input field with the placeholder text 'Введите ваш email' and the example text 'Например, ivan@gmail.com'.
- Input 3:** A text input field with the placeholder text 'Введите ваше сообщение' and the example text 'Здравствуйте...'.
- Button:** A red button with the text 'Отправить'.

The form is set against a light gray background with rounded corners. The consistent 35px spacing creates a clean and organized visual structure.

# Стилизуем .form-row

```
.form-row:nth-child(n+2) {  
  margin-top: 35px;  
}
```

Введите ваше имя

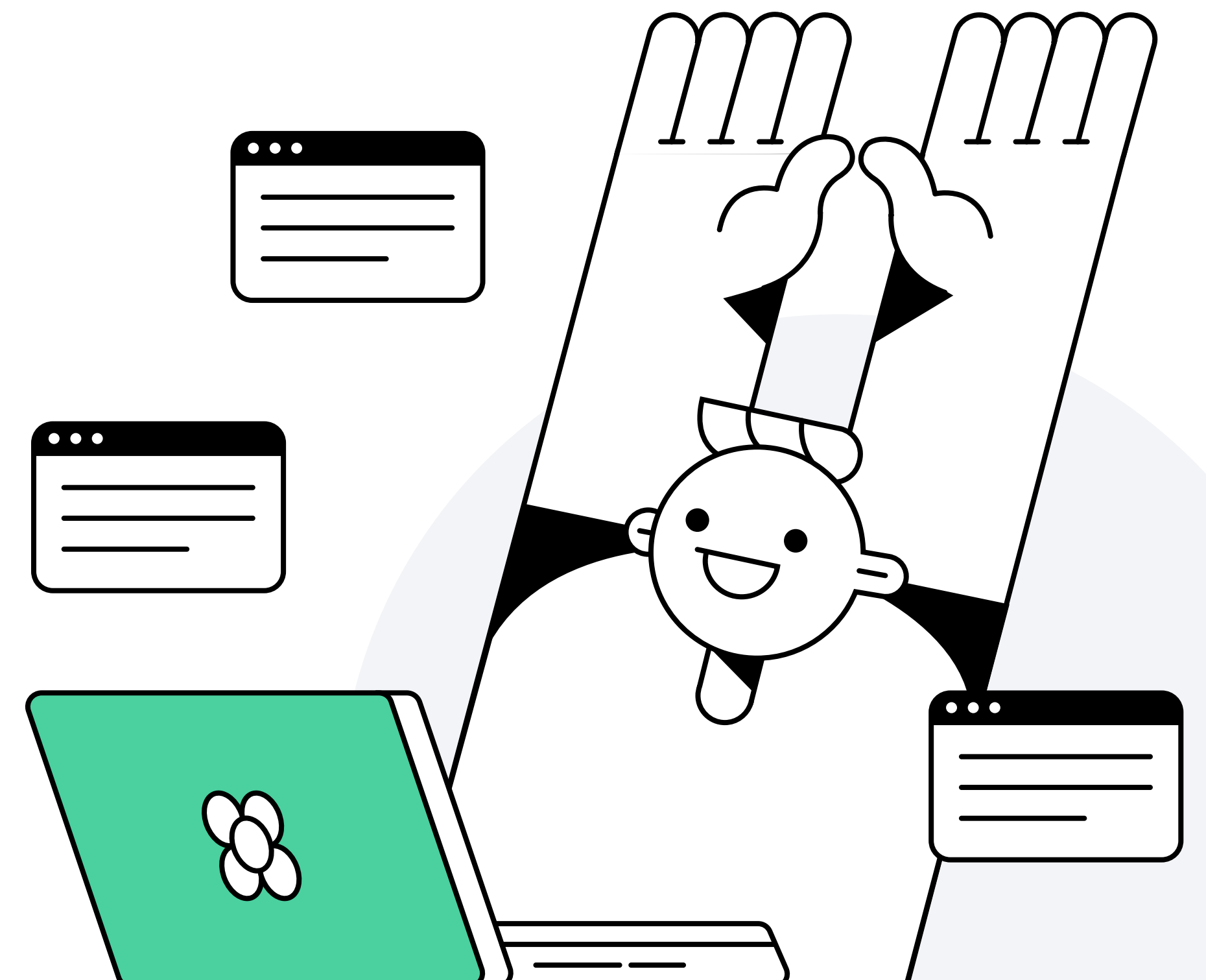
Введите ваш email

Введите ваше сообщение

Отправить

# Итоги

- 1 Элемент `button` предназначен для создания элемента управления, позволяющего пользователю запускать определённые действия

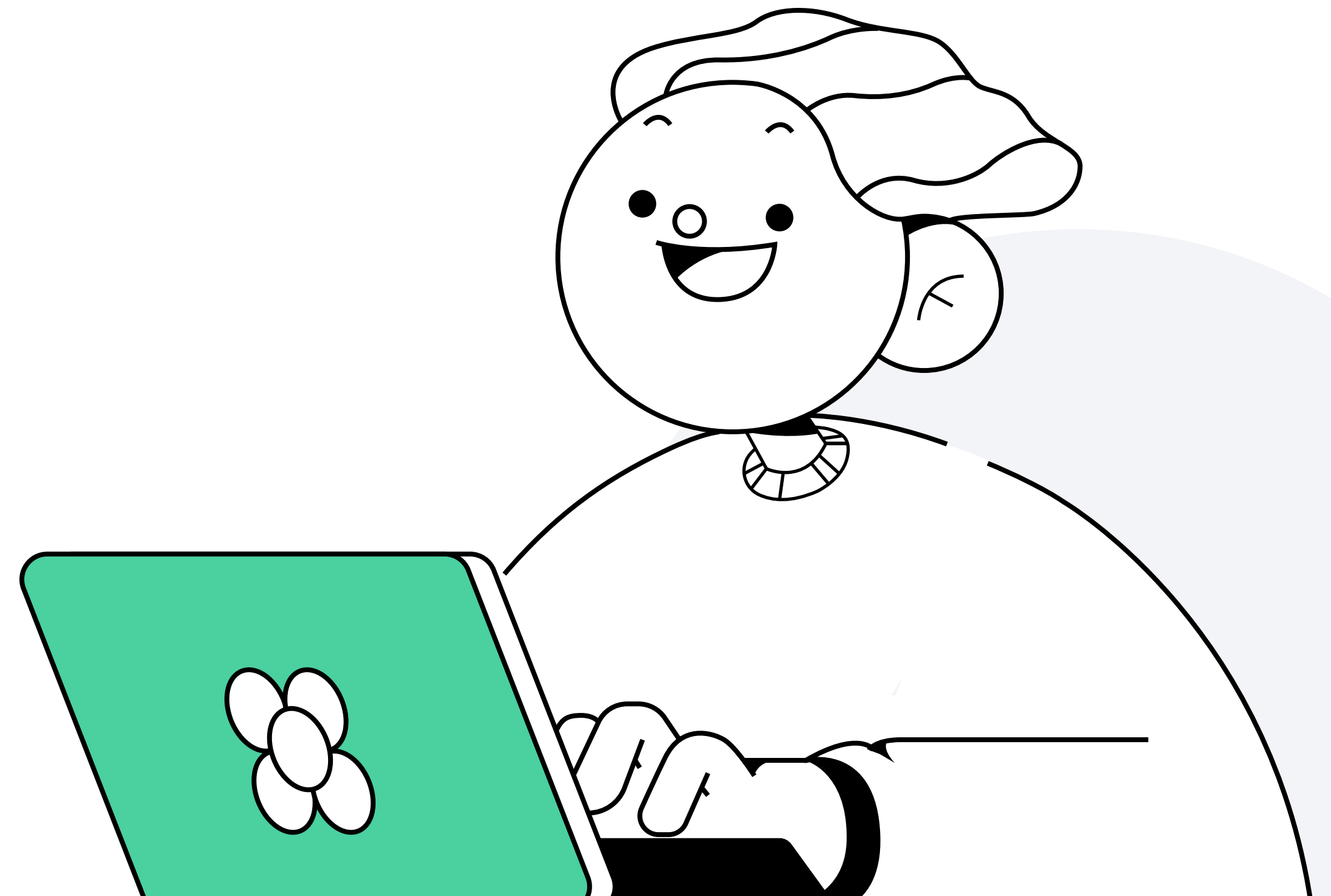


# Вёрстка форм. Поля для обязательного заполнения

Владимир Языков  
Frontend-разработчик

# Цели занятия

- 1 Узнать, как ответы отправляются на сервер
- 2 Научиться делать поля для ввода обязательными к заполнению
- 3 Научиться добавлять текстовое описание элемента формы



# Отправка данных

Введите ваше имя

Иван

Введите ваш email

ivan@gmail.com

Введите ваше сообщение

Привет!

Отправить

✗ Данные не отправляются

# Элемент form

```
<form class="form">
  <div class="form-row">
    <!-- поле для ввода имени -->
  </div>
  <div class="form-row">
    <!-- поле для ввода email -->
  </div>
  <div class="form-row">
    <!-- поле для ввода сообщения -->
  </div>
  <div class="form-row">
    <!-- кнопка -->
  </div>
</form>
```

# Атрибут required

Устанавливает поле формы обязательным для заполнения перед отправкой данных

```
<input type="text" required>
```

Ваше имя



Заполните это поле.

Google Chrome

Ваше имя



Заполните это поле.

Opera

Ваше имя

Пожалуйста, заполните это поле.

Firefox

Ваше имя

Это обязательное поле

Internet Explorer



# Результат

Введите ваше имя

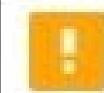
Например, Иван

Введите ваш email

Например, ivan@gmail.com

Введите ваше сообщение

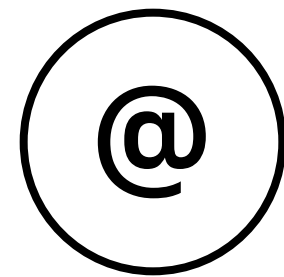
Здравствуйте...



Заполните это поле.

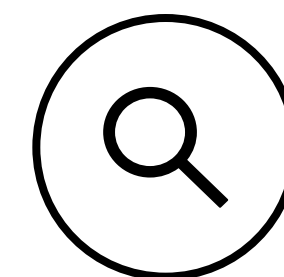
Отправить

# Значения type



**type="email"**

браузер будет ждать, что  
пользователь введёт email



**type="search"**

элемент становится полем  
ввода поискового запроса

# Изменим атрибут type

```
<input type="email" class="field" placeholder="Например, ivan@gmail.com" required>
```

Введите ваше имя

Иван

Введите ваш email

Это не электронный адрес



Адрес электронной почты должен содержать символ "@". В адресе "Это не электронный адрес" отсутствует символ "@".

Привет

Отправить

# Элемент label

Позволяет разметить текстовое описание элемента формы, которое связано с элементом `input`

Связать `label` и `input` можно двумя способами

① Вложить `input` в элемент `label`

```
<label>  
  Введите текст  
  <input type="text">  
</label>
```

② Использовать атрибут `for` для элемента `label` и `id` у элемента `input`

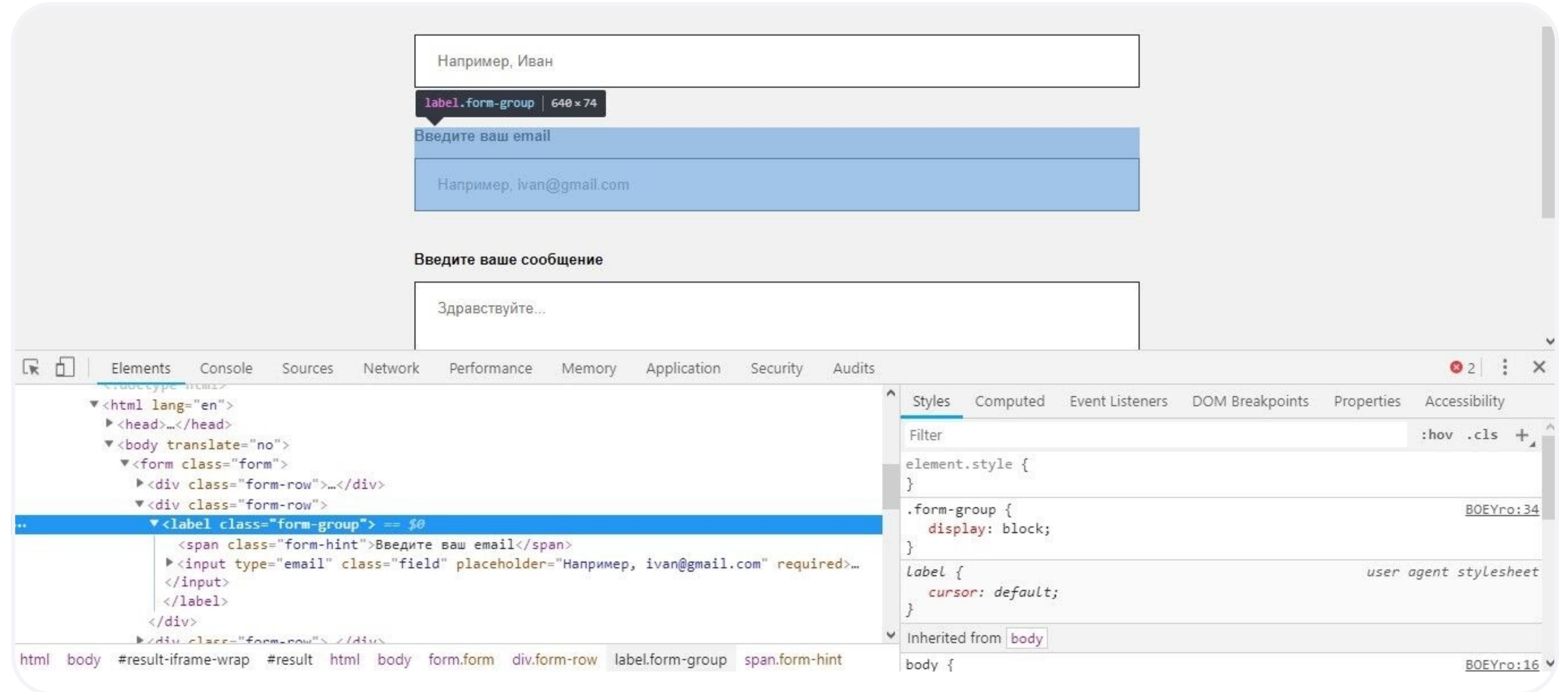
```
<label for="field-one">Введите текст</label>  
<input id="field-one" type="text">
```

# Добавим label

```
<div class="form-row">
  <label class="form-group">
    <span class="form-hint">Введите ваше имя</span>
    <input type="text" class="field" placeholder="Например, Иван" required>
  </label>
</div>
<div class="form-row">
  <label class="form-group">
    <span class="form-hint">Введите ваш email</span>
    <input type="text" class="field" placeholder="Например, ivan@gmail.com" required>
  </label>
</div>
<div class="form-row">
  <label class="form-group">
    <span class="form-hint">Введите ваше сообщение</span>
    <textarea class="field textarea" placeholder="Здравствуйтесь..." required></textarea>
  </label>
</div>
```

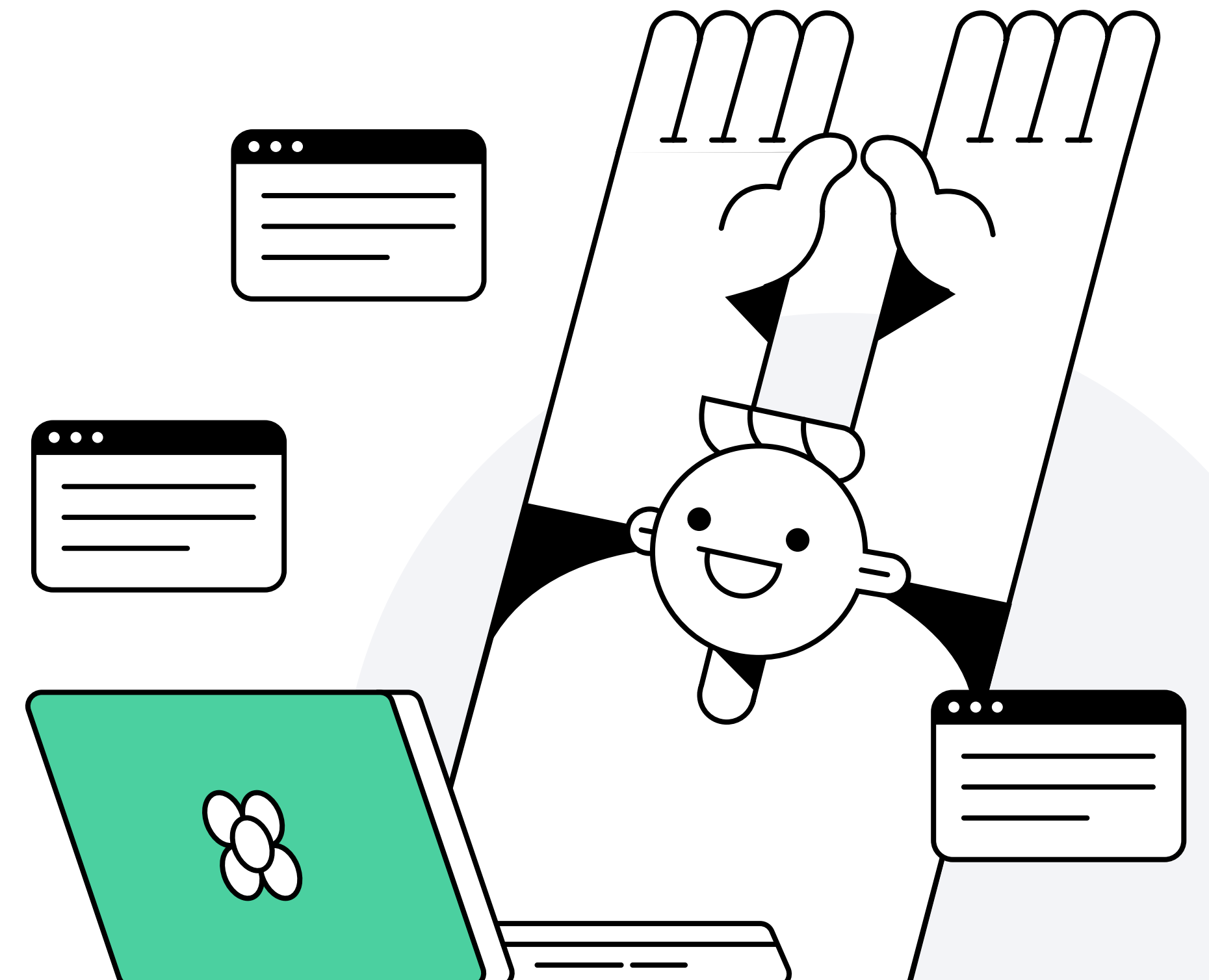
# Стилизуем label

```
.form-group {  
  display: block;  
}
```



# Итоги

- 1 Элемент `form` даёт возможность браузеру отправить введённые данные пользователя на сервер
- 2 Элемент `label` позволяет разметить текстовое описание элемента формы

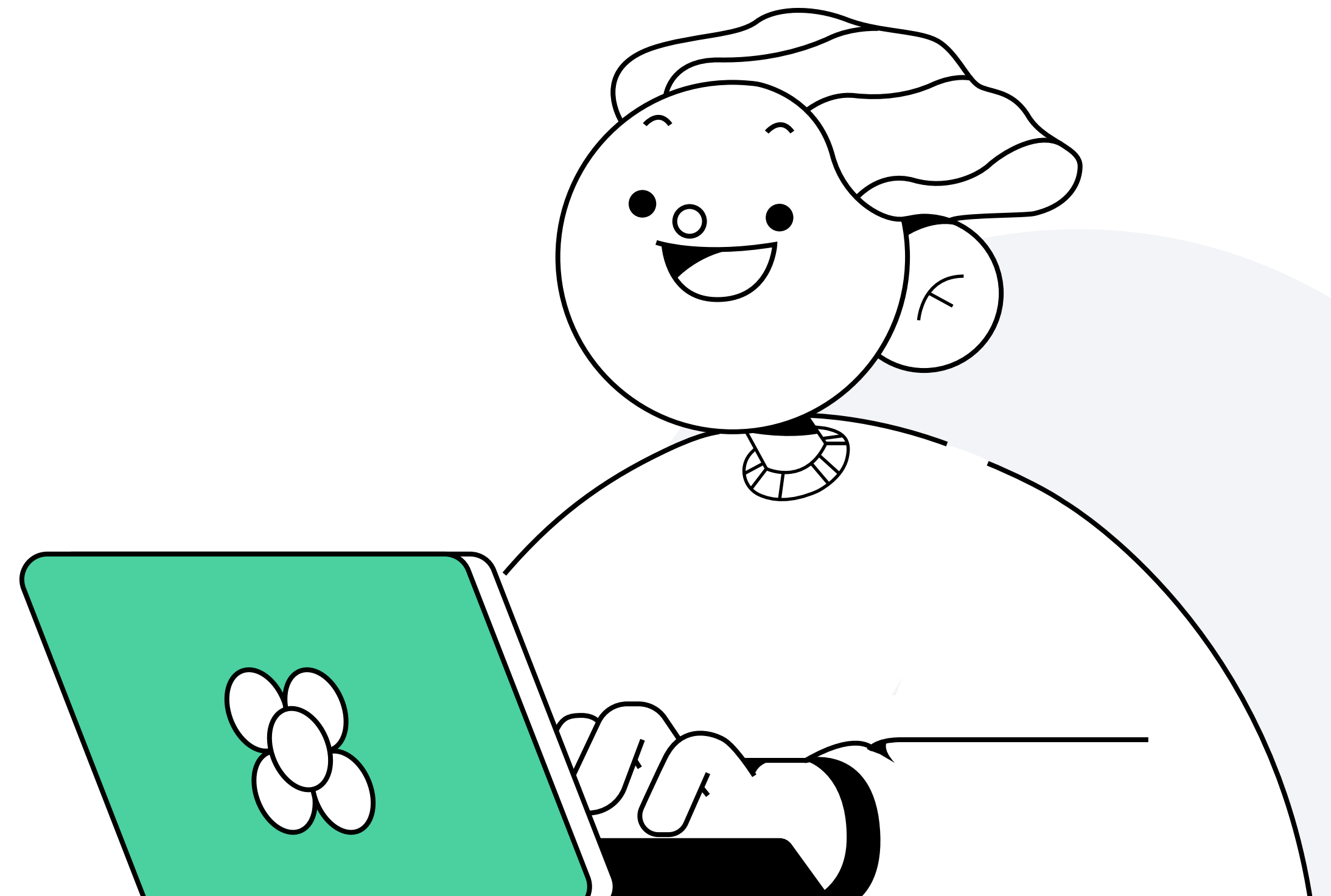


# Вёрстка форм. Выделение АКТИВНЫХ элементов

Владимир Языков  
Frontend-разработчик

# Цели занятия

- 1 Узнать разные способы выделения элемента
- 2 Научиться вёрстке выделения элемента



# Синяя обводка

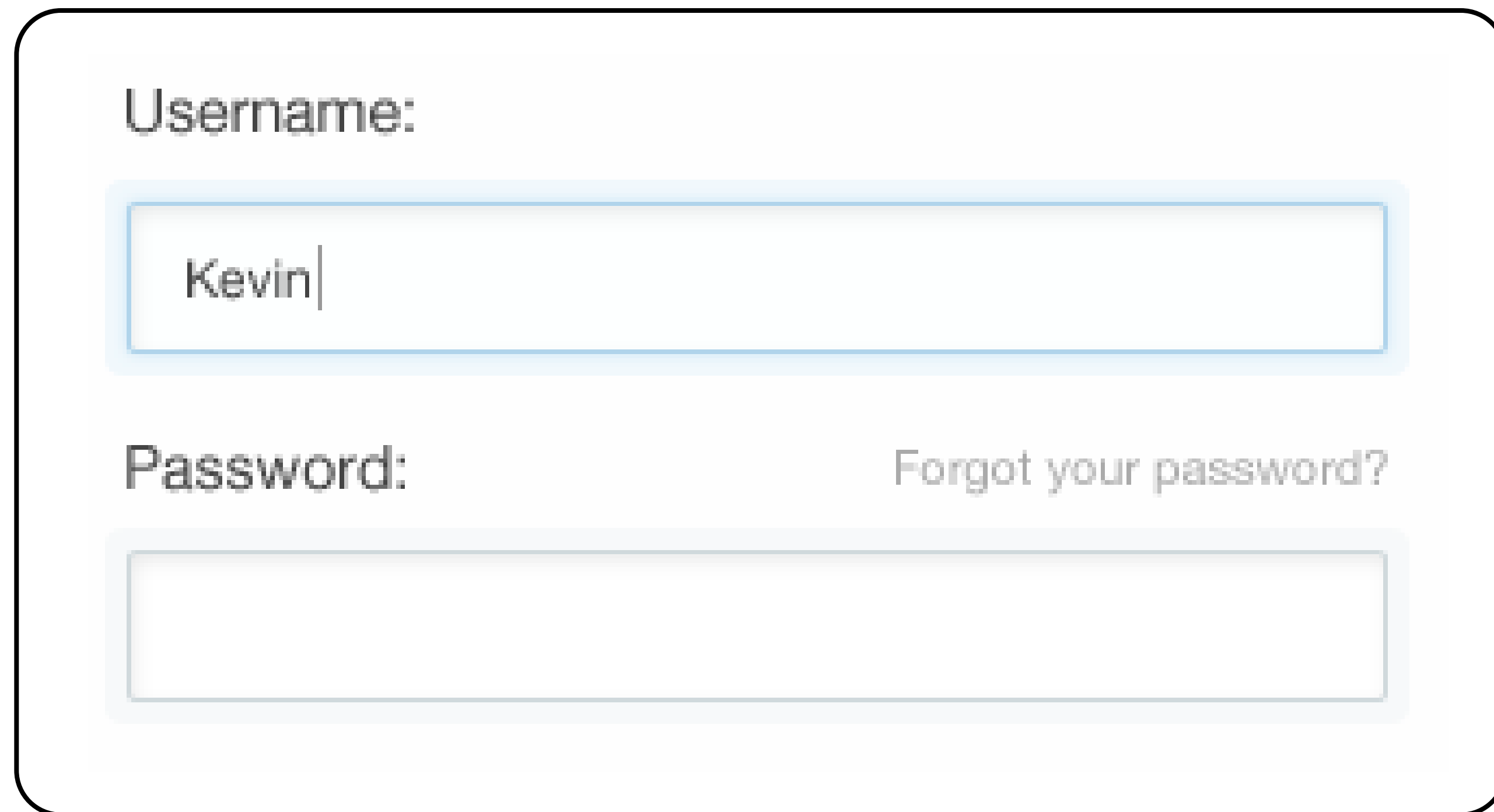
Введите ваше имя

Например, Иван

# Состояние :focus

Клавиша Tab переключает активные элементы.

Готовый к взаимодействию элемент называют «в состоянии фокуса»



Username:

Kevin|

Password:

[Forgot your password?](#)

The image shows a login form with two input fields. The first field, labeled 'Username:', contains the text 'Kevin|' and has a blue focus outline. The second field, labeled 'Password:', is empty and has a grey focus outline. A link 'Forgot your password?' is located to the right of the password field. The entire form is enclosed in a rounded rectangle with a black border.

# Свойство outline

Задаёт обводку вокруг элемента

Состоит из свойств:

⊞ outline-width

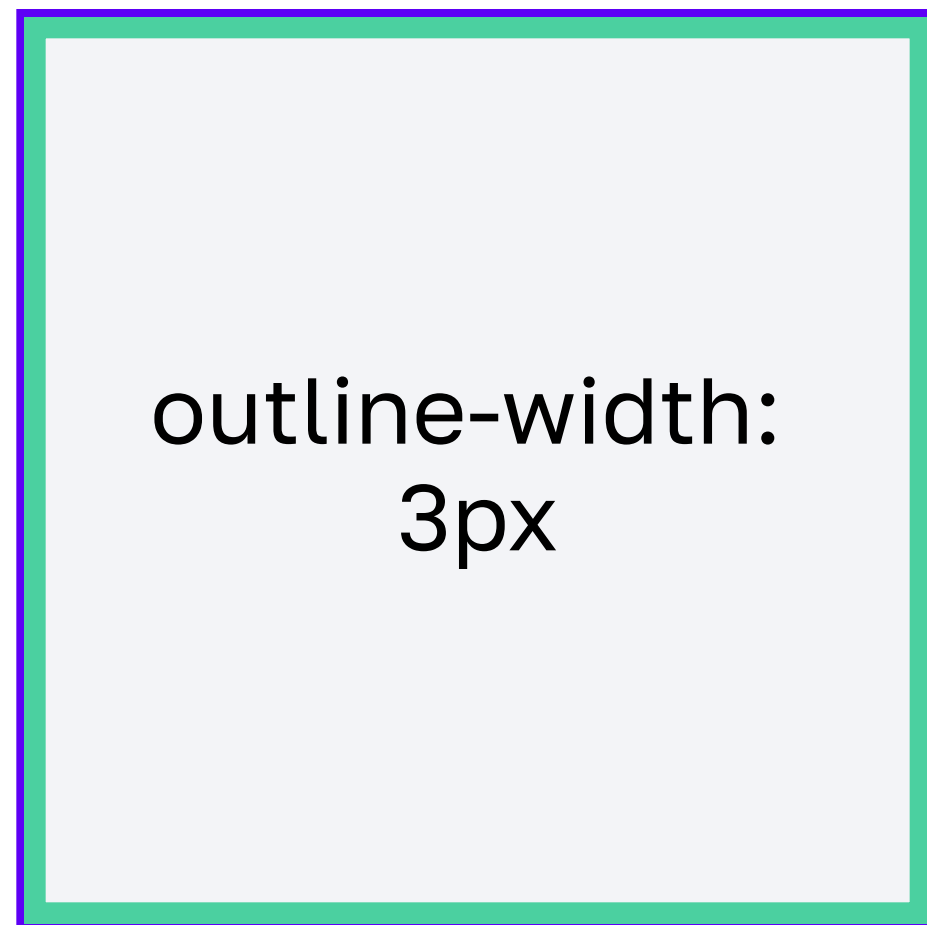
⋯ outline-style

— outline-color



# outline-width

Свойство `outline-width` задаёт ширину обводки.  
Задаётся в px

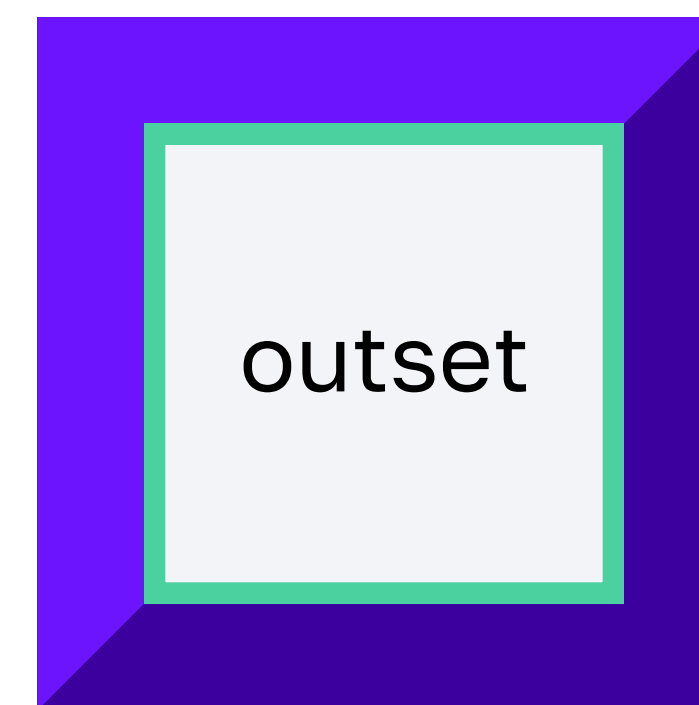
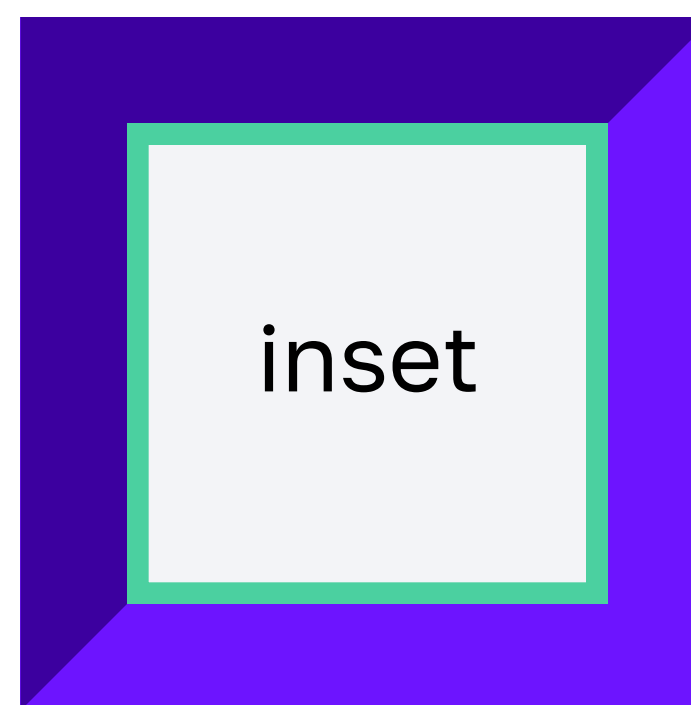
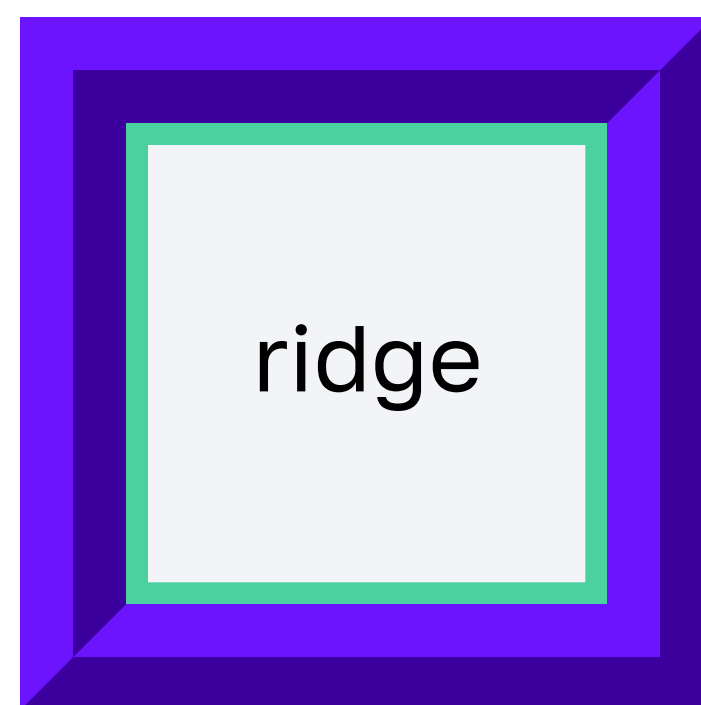
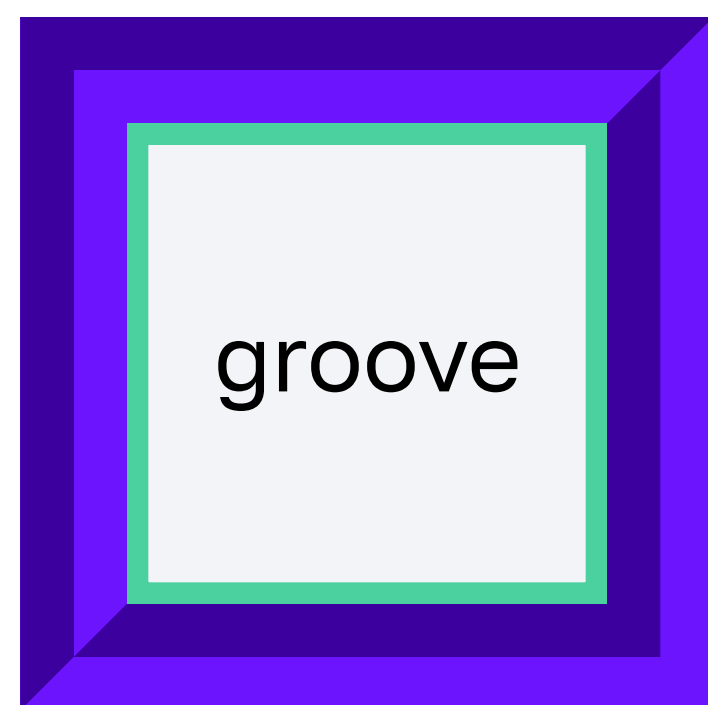
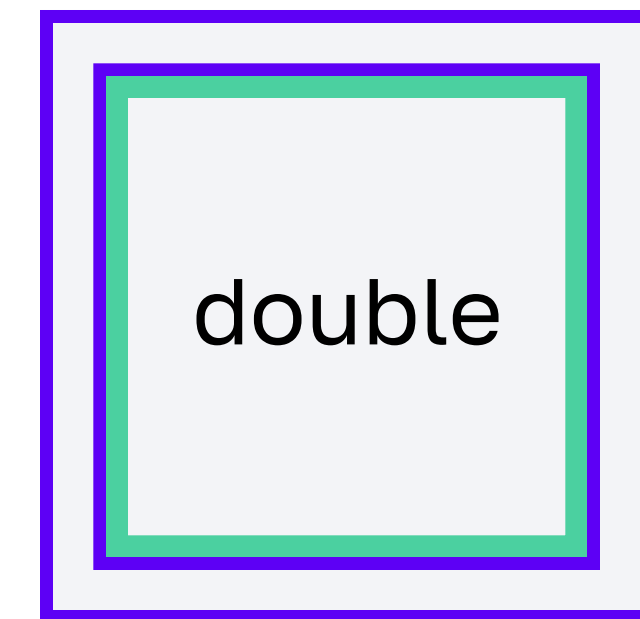
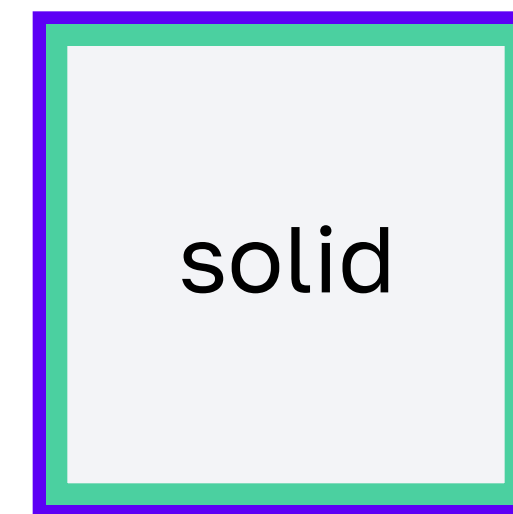
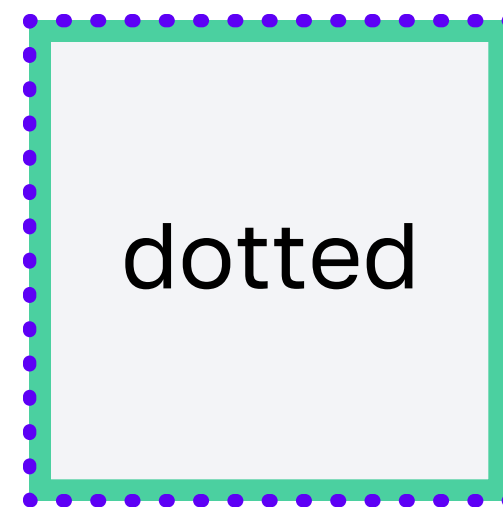
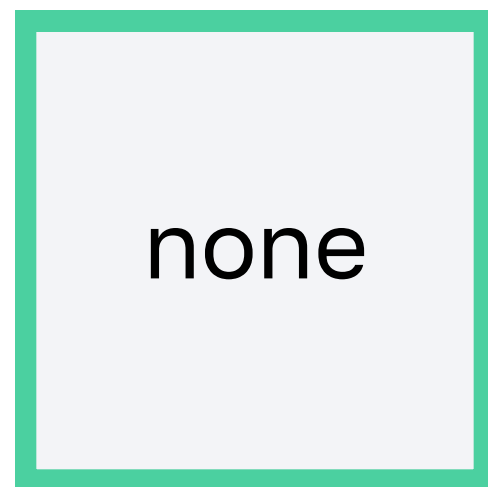


Свойство `border` обозначено зелёным

# outline-style

Свойство `outline-style` задаёт вид обводки.

Значения свойства

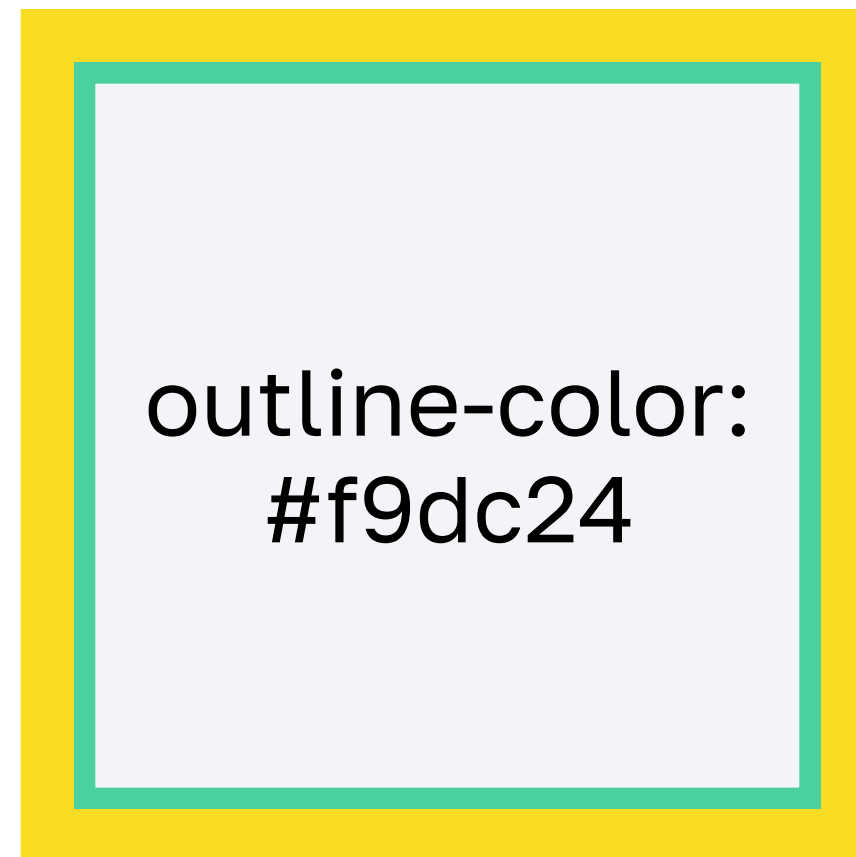
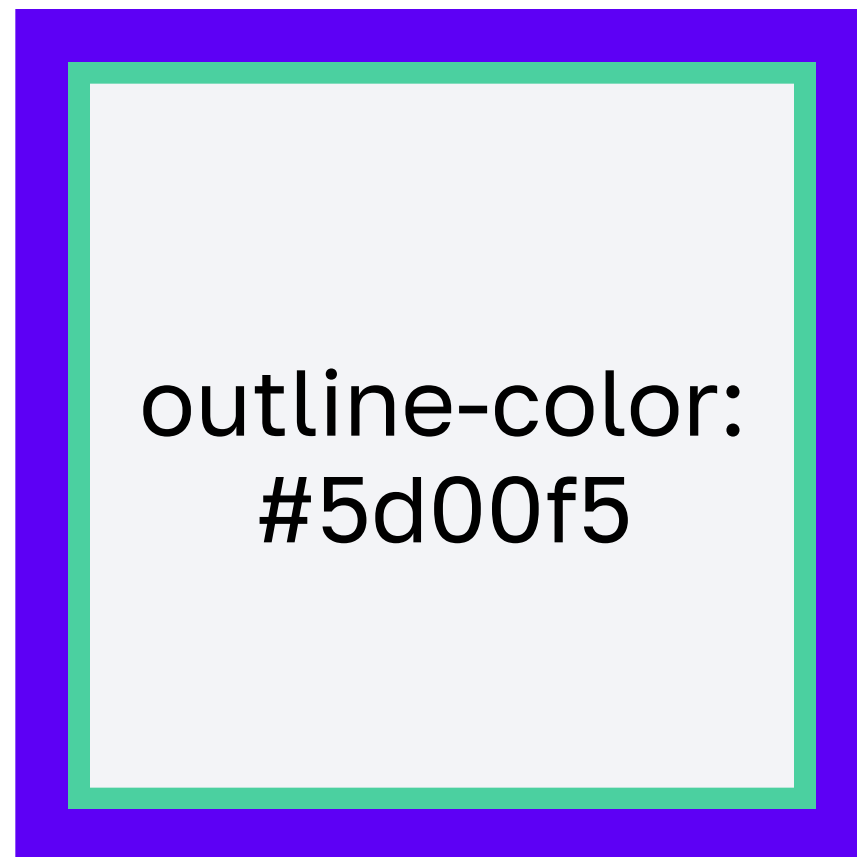


Свойство `border` обозначено зелёным

# outline-color

Свойство задаёт цвет обводки. В качестве значения  
МОЖНО ИСПОЛЬЗОВАТЬ шестнадцатеричный формат цвета

```
.button:focus {  
  outline: 3px solid #d2d2d2;  
}
```

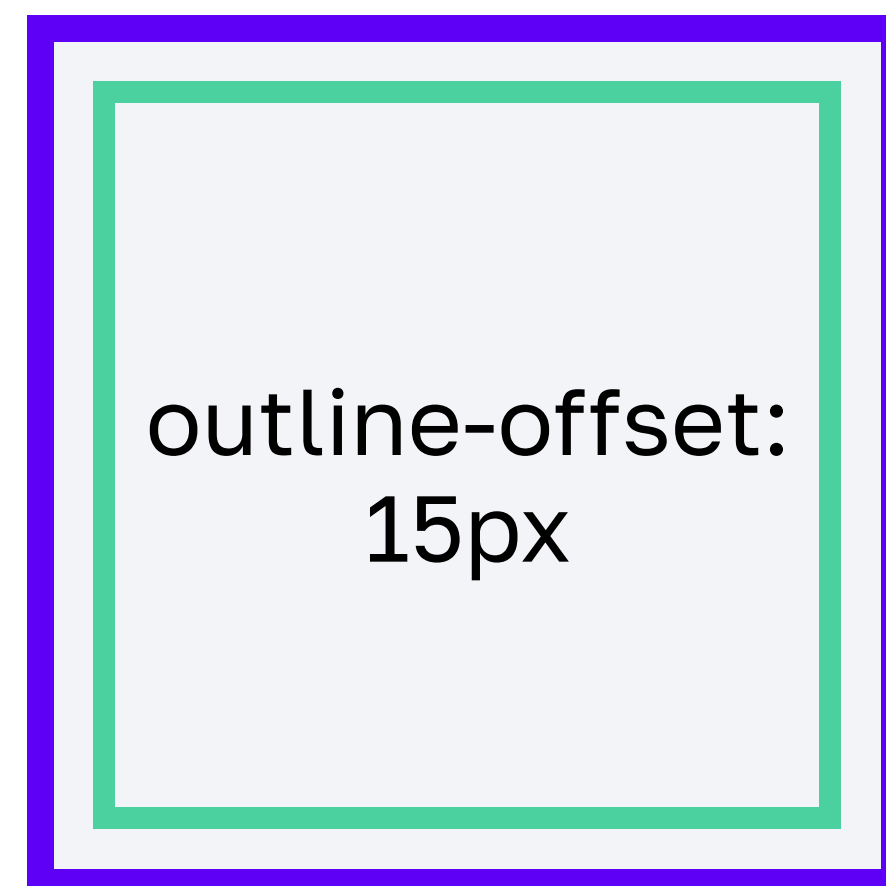
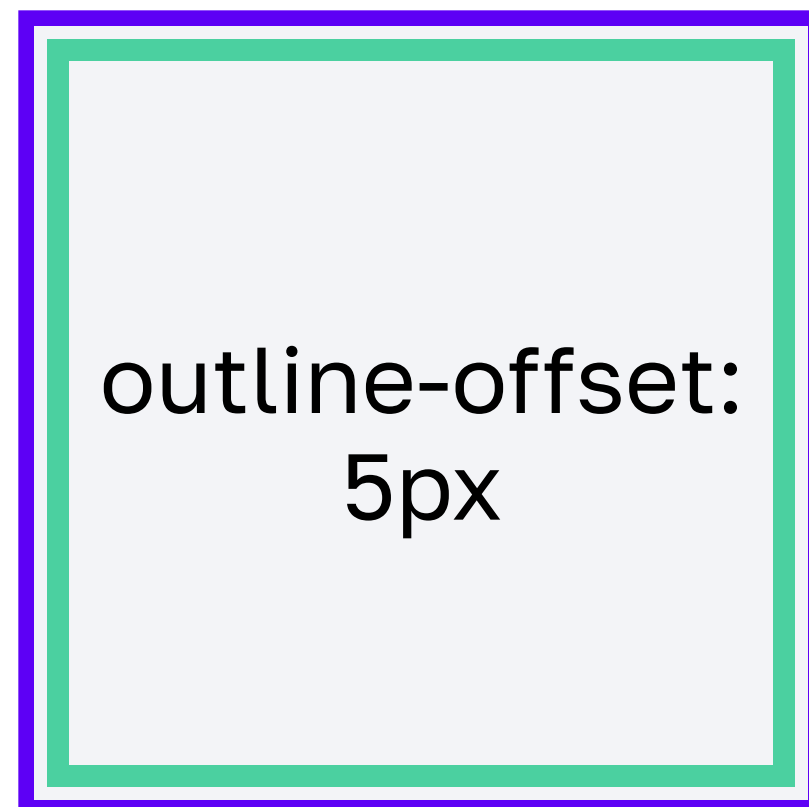


Свойство border обозначено зелёным

# outline-offset

Задаёт расстояние между обводкой outline и элементом.  
Значение для свойства указываются в px

```
.box {  
  outline: 15px solid #800080;  
  outline-offset: 15px;  
}
```



Свойство border обозначено зелёным

# Стилизация :focus

```
.field:focus {  
  outline: 2px solid #c53b2d;  
  outline-offset: 5px;  
}
```

```
.button:focus {  
  outline: 2px solid #c53b2d;  
  outline-offset: 5px;  
}
```

Введите ваш email

Например, ivan@gmail.com

Отправить

# Список селекторов

## Объединение повторяющихся правил в CSS

```
h1 { font-family: sans-serif; }  
h2 { font-family: sans-serif; }  
h3 { font-family: sans-serif; }
```

Этот код можно переписать, создав список селекторов

```
h1, h2, h3 { font-family: sans-serif; }
```

# Состояние :hover

При :hover элемент меняет свойства при наведении на него курсора

```
.button:hover {  
  background-color: #dddddd;  
}  
  
a:hover {  
  color: #dddddd;  
}
```



# Состояние :hover

Смена цвета кнопки  
при наведении мыши:

```
.button:hover {  
  background-color: #c53b2d;  
}
```



# Итоги

- 1 С помощью псевдокласса `:hover` можно задать свойства, которые будут применяться только при наведении мышкой на элемент
- 2 С помощью псевдокласса `:focus` можно задать свойства, которые будут применяться, только когда браузер установил фокус на элементе

